INTRODUCTION TO C++

WORKSHEET 3

Submitted By: **Sadhabi Dev**
Level : 4
Student ID : 23085140
Cyber Security and Digital Forensics

QUESTION NO.1

Create a Time class to store hours and minutes. Implement:

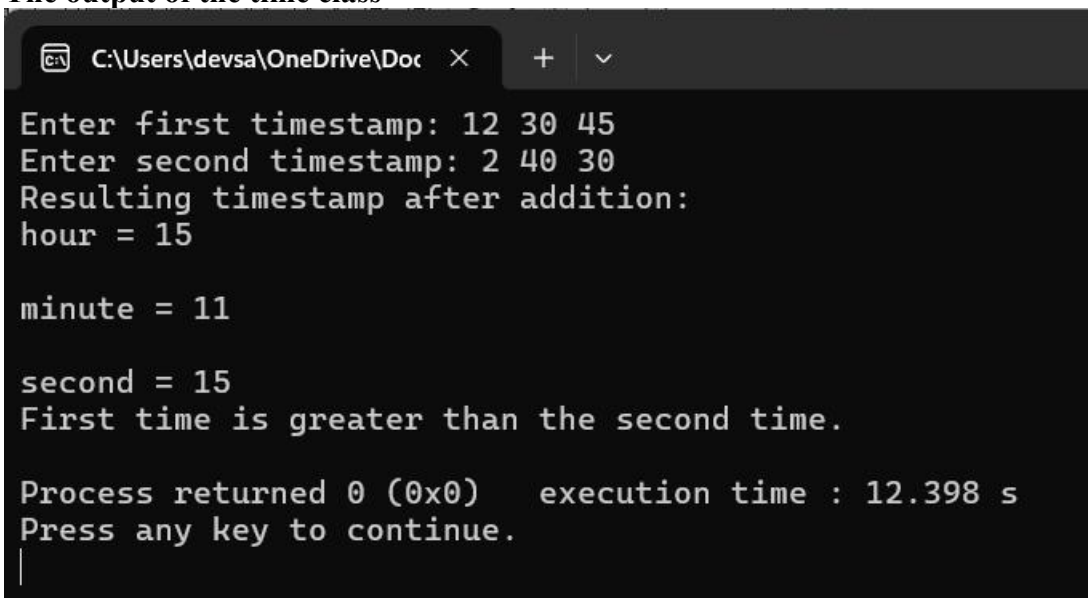
- Overload the + operator to add two Time objects
- Overload the > operator to compare two Time objects
- Handle invalid time (>24 hours or >60 minutes) by throwing a custom exception

SOLUTION:

```
#include <iostream>
using namespace std;

class Time {
private:
   int h;
   int m;
   int s;

public:
   Time() {
      h = 0;
      m = 0;
      s = 0;
   }

   Time(int hour, int minute, int second) : h(hour), m(minute), s(second) {
      if (h > 24 || m > 60 || s > 60) {
         throw "Invalid time: hour should be <= 24, minute and second should be <=
60";
      }
   }

   Time operator +(Time t) {
      Time temp;
      int total_seconds = s + t.s;
      int newminute = total_seconds / 60;
      temp.s = total_seconds % 60;
      int total_minutes = m + t.m + newminute;
      int newhour = total_minutes / 60;
      temp.m = total_minutes % 60;
      temp.h = h + t.h + newhour;
      if (temp.h >= 24) {
         temp.h = temp.h % 24;
      }
      if (temp.m >= 60) {
         temp.m = temp.m % 60;
      }
      return temp;
   }
```

```
    bool operator >(Time t) {
        if (h > t.h) {
            return true;
        } else if (h == t.h && m > t.m) {
            return true;
        } else if (h == t.h && m == t.m && s > t.s) {
            return true;
        }
        return false;
    }

    void display() {
        cout << "\nhour = " << h << endl;
        cout << "\nminute = " << m << endl;
        cout << "\nsecond = " << s << endl;
    }
};

int main() {
    int a, b, c;
    int d, e, f;

    try {
        cout << "Enter first timestamp: ";
        cin >> a >> b >> c;

        cout << "Enter second timestamp: ";
        cin >> d >> e >> f;

        Time t1(a, b, c);
        Time t2(d, e, f);

        Time result = t1 + t2;

        cout << "Resulting timestamp after addition: ";
        result.display();

        if (t1 > t2) {
            cout << "First time is greater than the second time." << endl;
        } else {
            cout << "Second time is greater than or equal to the first time." << endl;
        }

    } catch (const char* msg) {
        cout << msg << endl;
    }

    return 0;
}
```

The output of the time class



QUESTION NO.2

Create a base class Vehicle and two derived classes Car and Bike:

- Vehicle has registration number and color
- Car adds number of seats
- Bike adds engine capacity
- Each class should have its own method to write its details to a file
- Include proper inheritance and method overriding

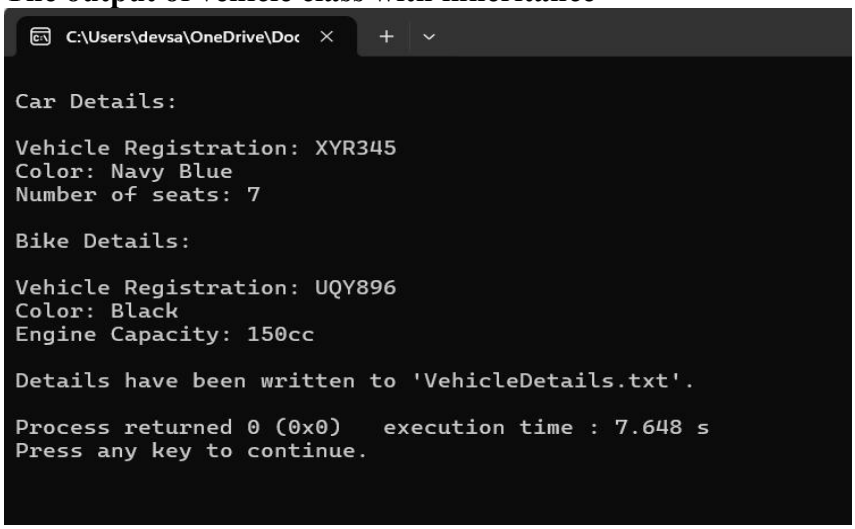
SOLUTION:

```
#include <iostream>
#include <fstream>
using namespace std;

class Vehicle {
public:
   string regNo;
   string color;

   Vehicle(string reg, string col) {

      regNo = reg;
      color = col;
   }

   void displayData() {
```

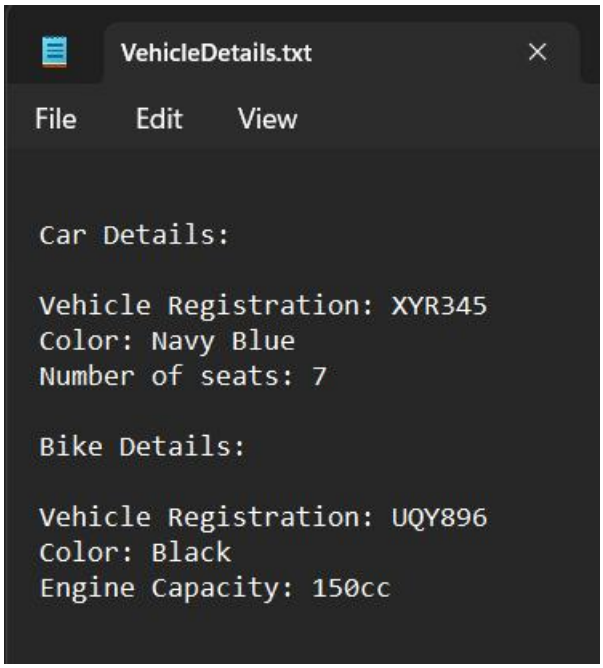
```
        cout << "\nVehicle Registration: " << regNo << endl;
        cout << "Color: " << color << endl;
    }

    void writeToFile(ofstream &outFile) {

        outFile << "\nVehicle Registration: " << regNo << endl;
        outFile << "Color: " << color << endl;
    }
};

class Car : public Vehicle {
public:
    int noOfSeats;

    Car(string reg, string col, int seats) : Vehicle(reg, col) {

        noOfSeats = seats;
    }

    void displayData() {

        cout << "\nCar Details:" << endl;
        Vehicle::displayData();
        cout << "Number of seats: " << noOfSeats << endl;
    }

    void writeToFile(ofstream &outFile) {

        outFile << "\nCar Details:" << endl;
        Vehicle::writeToFile(outFile);
        outFile << "Number of seats: " << noOfSeats << endl;
    }
};

class Bike : public Vehicle {

public:
    int engineCapacity;

    Bike(string reg, string col, int capacity) : Vehicle(reg, col) {

        engineCapacity = capacity;
    }

    void displayData() {

        cout << "\nBike Details:" << endl;
        Vehicle::displayData();
        cout << "Engine Capacity: " << engineCapacity << "cc" << endl;
```

```
    }

    void writeToFile(ofstream &outFile) {

        outFile << "\nBike Details:" << endl;
        Vehicle::writeToFile(outFile);
        outFile << "Engine Capacity: " << engineCapacity << "cc" << endl;
    }
};

int main() {

    Car c1("XYR345", "Navy Blue", 7);
    Bike b1("UQY896", "Black", 150);

    c1.displayData();
    b1.displayData();

    ofstream outFile("VehicleDetails.txt");
    if (outFile.is_open()) {

        c1.writeToFile(outFile);
        b1.writeToFile(outFile);
        outFile.close();

        cout << "\nDetails have been written to 'VehicleDetails.txt'." << endl;

    } else {

        cout << "Error opening file!" << endl;
    }

    return 0;
}
```

The output of vehicle class with inheritance



The record has been added to the text file



QUESTION NO.3

Create a program that:

- Reads student records (roll, name, marks) from a text file
- Throws an exception if marks are not between 0 and 100
- Allows adding new records with proper validation
- Saves modified records back to file

SOLUTION:

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

class Student {
public:
    int rollNo;
    char name[50];
    double marks;

    void inputData() {
        cout << "\nEnter Roll No: ";
        cin >> rollNo;
        cout << "Enter Name: ";
        cin.ignore();
        cin.getline(name, 50);
        cout << "Enter Marks (0-100): ";
        cin >> marks;
    }
```

```
    void displayData() {
        cout << "\n---------------------------------------" << "\n";
        cout << "Roll No: " << rollNo << "\n";
        cout << "Name: " << name << "\n";
        cout << "Marks: " << marks << "\n";
        cout << "---------------------------------------" << "\n";
    }
};

void checkMarks(double m) {
    if (m < 0 || m > 100) {
        throw "Marks should be between 0 and 100.";
    }
}

void readFile(Student students[], int& count, string filename) {
    ifstream file(filename);
    if (!file.is_open()) {
        cerr << "\nError: Unable to open file.\n";
        return;
    }

    while (file >> students[count].rollNo >> students[count].name >>
students[count].marks) {
        count++;
    }

    file.close();
}

void saveFile(Student students[], int count, string filename) {
    ofstream file(filename);
    if (!file.is_open()) {
        cerr << "\nError: Unable to open file.\n";
        return;
    }

    for (int i = 0; i < count; ++i) {
        file << students[i].rollNo << " " << students[i].name << " " << students[i].marks
<< endl;
    }

    file.close();
}

void addStudent(Student students[], int& count) {
    Student newStudent;
    newStudent.inputData();
```

```
    try {
        checkMarks(newStudent.marks);
        students[count] = newStudent;
        count++;
        cout << "\nStudent record added successfully!\n";
    } catch (char* e) {
        cout << "\n" << e << "\n";
    }
}

void showMenu() {
    cout << "\n***********************************************";
    cout << "\n                Main Menu";
    cout << "\n***********************************************";
    cout << "\n1) Add Student Record";
    cout << "\n2) Save and Exit";
    cout << "\n3) Show Student Records";
    cout << "\n===============================================";
    cout << "\nEnter your choice: ";
}

void showRecords(Student students[], int count) {
    if (count == 0) {
        cout << "\nNo student records to display.\n";
        return;
    }
    cout << "\n*********************************************";
    cout << "\n             Saved Student Records";
    cout << "\n*********************************************";
    for (int i = 0; i < count; ++i) {
        students[i].displayData();
    }
    cout << "=============================================\n";
}

void menu() {
    Student students[100];
    int count = 0;
    string filename = "students.txt";

    readFile(students, count, filename);

    int choice;
    while (true) {
        showMenu();
        cin >> choice;

        if (choice == 1) {
            addStudent(students, count);
        } else if (choice == 2) {
```

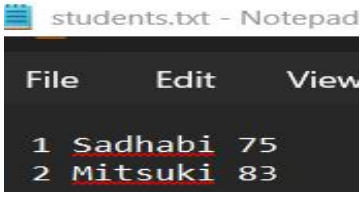
```
            saveFile(students, count, filename);
            cout << "\nRecords saved successfully.\n";
            showRecords(students, count);
            cout << "\nExiting program...\n";
            break;
        } else if (choice == 3) {
            showRecords(students, count);
        } else {
            cout << "\nInvalid choice! Please try again.\n";
        }
    }
}

int main() {
    menu();
    return 0;
}
```

The output of the student record system

```
*********************************************
                Main Menu
*********************************************
1) Add Student Record
2) Save and Exit
3) Show Student Records
=============================================
Enter your choice: 3

No student records to display.

*********************************************
                Main Menu
*********************************************
1) Add Student Record
2) Save and Exit
3) Show Student Records
=============================================
Enter your choice: 1

Enter Roll No: 1
Enter Name: Sadhabi
Enter Marks (0-100): 75

Student record added successfully!

*********************************************
                Main Menu
*********************************************
1) Add Student Record
2) Save and Exit
3) Show Student Records
=============================================
Enter your choice: 1

Enter Roll No: 2
Enter Name: Mitsuki
Enter Marks (0-100): 83

Student record added successfully!
```

```
*****************************************
                Main Menu
*****************************************
1) Add Student Record
2) Save and Exit
3) Show Student Records
=========================================
Enter your choice: 3

*****************************************
             Saved Student Records
*****************************************
-----------------------------------
Roll No: 1
Name: Sadhabi
Marks: 75
-----------------------------------

-----------------------------------
Roll No: 2
Name: Mitsuki
Marks: 83
-----------------------------------
=========================================

*****************************************
                Main Menu
*****************************************
1) Add Student Record
2) Save and Exit
3) Show Student Records
=========================================
Enter your choice: 2

Records saved successfully.

*****************************************
             Saved Student Records
*****************************************
-----------------------------------
Roll No: 1
Name: Sadhabi
Marks: 75
-----------------------------------

-----------------------------------
Roll No: 2
Name: Mitsuki
Marks: 83
-----------------------------------
=========================================

Exiting program...

Process returned 0 (0x0)   execution time : 164.238 s
Press any key to continue.
```

The record have been added to the .txt file



Rechecking if the results have been added to the system or not

```
*****************************************
                Main Menu
*****************************************
1) Add Student Record
2) Save and Exit
3) Show Student Records
=========================================
Enter your choice: 3

*****************************************
             Saved Student Records
*****************************************
-----------------------------------------
Roll No: 1
Name: Sadhabi
Marks: 75
-----------------------------------------

-----------------------------------------
Roll No: 2
Name: Mitsuki
Marks: 83
-----------------------------------------
=========================================

*****************************************
                Main Menu
*****************************************
1) Add Student Record
2) Save and Exit
3) Show Student Records
=========================================
Enter your choice: 2

Records saved successfully.

*****************************************
             Saved Student Records
*****************************************
-----------------------------------------
Roll No: 1
Name: Sadhabi
Marks: 75
-----------------------------------------

-----------------------------------------
Roll No: 2
Name: Mitsuki
Marks: 83
-----------------------------------------
=========================================

Exiting program...

Process returned 0 (0x0)   execution time : 7.946 s
Press any key to continue.
```

My Github Link: https://github.com/sd6012/cpp_Assignment